

Alpamayo 1.5

A Visual Study Guide to NVIDIA's Reasoning VLA for Autonomous Driving

Architecture, the **Cosmos Reason 2** backbone, and the **AlpaSim** closed-loop simulator that puts it all to the test.

Prepared as a self-study companion to the
Alpamayo Explore reproduction project.

Contents

The Big Picture

What problem does Alpamayo solve?

Three pieces, one stack

Why "reasoning" VLA matters

Alpamayo 1.5 — The Driver

Architecture at a glance

Inputs, outputs, and the chain of causation

How Alpamayo 1.5 differs from Alpamayo 1 (R1)

Training data and benchmarks

Cosmos Reason 2 — The Reasoning Brain

An 8B reasoning VLM for physical AI

Architecture: ViT + dense transformer

Why Alpamayo uses Cosmos as its backbone

AlpaSim — The Closed-Loop Testbed

Why simulate, and what kind of simulation?

Microservice architecture

NuRec scenes: real driving made replayable

Outputs: video, ASL, metrics

How They Fit Together — A Single Driving Step

A Real Run, End to End

References

1. The Big Picture

1.1 What problem does Alpamayo solve?

Self-driving cars have improved dramatically on *frequent* situations — highway lane keeping, freeway following, simple intersections. The hard part remaining is the **long tail**: rare scenarios that human drivers handle by reasoning, not reflex. A construction zone where cones encroach into the lane. A pedestrian stepping off the curb between parked cars. A right-turning bus that suddenly stops, blocking your lane. These cases are where most autonomous-vehicle programs lose their nerve and disengage.

NVIDIA's **Alpamayo** family of models is one answer to this: a *reasoning* Vision-Language-Action model that, alongside its trajectory output, emits a written *chain of causation* — short text explaining what it observed, what it inferred, and why it chose the action it did. This makes the model's decisions inspectable in a way earlier end-to-end driving networks never were.

1.2 Three pieces, one stack

To understand Alpamayo 1.5 in isolation is impossible — the model relies on two other open NVIDIA components, and the only way to actually *run* it on real driving data is through a third. The three pieces:

THE DRIVER

Alpamayo 1.5

10 B-parameter VLA. Sees cameras + maps, outputs trajectory + reasoning text.

THE REASONING BRAIN

Cosmos Reason 2 — 8 B

Vision-language model embedded inside Alpamayo as the perception/reasoning backbone.

THE TESTBED

AlpaSim

Closed-loop simulator that replays real driving scenes and lets a driver model take control.

THE SCENES

NuRec dataset

USDZ reconstructions of 900+ real driving clips (20 s each), used as inputs to AlpaSim.

Read the chapters in order for the cleanest mental model: driver first, then the brain inside it, then the simulator that exercises it on real scenes.

1.3 Why "reasoning" VLA matters

A traditional end-to-end driving network is a function: `cameras → trajectory`. There is no intermediate language layer. If the model swerves to avoid an obstacle that wasn't actually there, you cannot easily ask it *why*; you can only retrain. A reasoning VLA is a function with a witness: `cameras → (trajectory, written reasoning)`. The reasoning text is generated alongside the action and is causally linked to it during training, so the same internal computation that decides the action also produces the explanation.

Concrete example from a real Alpamayo run on a construction-zone scene (extracted from the model's `rollout.asl` binary log):

Reasoning text emitted at one timestep:

"Slow down to create a gap for a merge into the left lane because our current right lane narrows / ends ahead and a car is traveling alongside in our left lane, blocking an immediate merge."

Notice the structure: action ("slow down to create a gap"), cause #1 ("right lane narrows"), cause #2 ("car alongside blocks merge"), and an implicit plan ("merge once the gap forms"). This is the unit of output that, scaled across hundreds of timesteps, makes the model's policy auditable.

2. Alpamayo 1.5 – The Driver

2.1 Architecture at a glance

Alpamayo 1.5 is a modular VLA: a vision-language backbone produces a latent representation of the scene plus a reasoning text stream, and a separate diffusion-based decoder turns that representation into a future trajectory. The two halves are trained jointly so that what the model says and what it *does* are tied together.

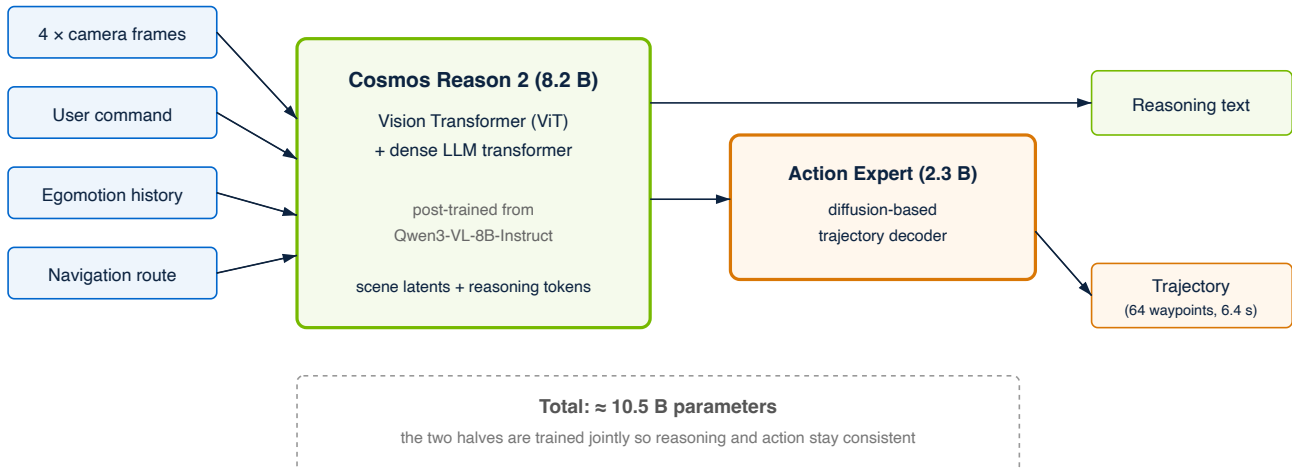


Figure 2.1 — Alpamayo 1.5 high-level architecture. The backbone is Cosmos Reason 2; the action decoder is a diffusion model that produces a 64-waypoint future trajectory.

2.2 Inputs, outputs, and the chain of causation

INPUTS

- **Multi-camera RGB:** by default 4 cameras — front-wide (120°), front-tele (30°), cross-left (120°), cross-right (120°). Native resolution 1080×1920, downsampled to 320×576 for the model. The model takes 4 frames per camera (a 0.4 s history at 10 Hz).
- **Text command:** a free-form user instruction or navigation guidance (e.g., "go straight," "take the next right"). Alpamayo 1.5 added flexible support for navigation commands.
- **Egomotion history:** the past 16 waypoints (1.6 s at 10 Hz) of the ego vehicle's pose — 3D translation plus a 3×3 rotation matrix.

OUTPUTS

- **Reasoning text:** a chain-of-causation trace generated alongside the trajectory — a short natural-language explanation that NVIDIA labels with `<think>` tags during training so the reasoning is grounded in the action that follows.
- **Trajectory:** 64 future waypoints (6.4 s at 10 Hz) in the ego vehicle's coordinate frame. Internally the model emits a sequence of unicycle-model dynamic actions (acceleration + curvature in BEV space) that are then decoded into pose waypoints.

2.3 How Alpamayo 1.5 differs from Alpamayo 1 (R1)

Alpamayo R1 (the December 2025 release) and Alpamayo 1.5 (March 2026) share the modular VLA shape — Cosmos backbone + diffusion action decoder — but the 1.5 release made several improvements visible from the model card:

- **RL post-training:** a reinforcement-learning stage on top of the supervised model, tightening reasoning-action consistency.
- **Navigation guidance:** explicit support for route/command inputs the model can ground its plans against.
- **Flexible camera count:** the default is 4 cameras but the architecture no longer requires it.
- **User question answering:** the model can be queried with natural-language questions about the scene (a side-channel on top of the driving task).
- **Better closed-loop score:** the published AlpaSim benchmark over 910 NuRec scenarios moved from 0.73 ± 0.01 (R1) to 0.81 ± 0.01 (1.5).

2.4 Training data and benchmarks

Aspect	Alpamayo 1.5
Total parameters	≈ 10.5 B (8.2 B backbone + 2.3 B action decoder)
Driving footage	$\approx 80,000$ hours, multi-camera
Images	> 1 billion
Reasoning traces	≈ 3 million chain-of-causation traces (mostly auto-labeled by VLMs, curated with human-in-the-loop)
Trajectory data	10 Hz sampling across all 80 k hours
Closed-loop benchmark (AlpaSim)	0.81 ± 0.01 over 910 NuRec scenarios
Open-loop benchmark	minADE @ 6.4 s = 1.11 m on PhysicalAI-AV (937 challenging samples)
Hardware to run inference	1 GPU, ≥ 24 GB VRAM (RTX 3090/4090/A5000) — tested on H100
License (weights)	NVIDIA non-commercial; inference code Apache 2.0

Source: [Alpamayo-1.5-10B model card](#); [arXiv 2511.00088](#) (Alpamayo R1 paper).

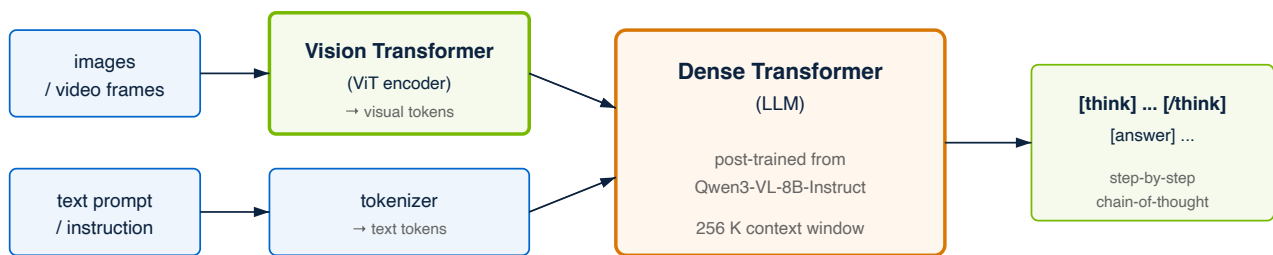
3. Cosmos Reason 2 — The Reasoning Brain

3.1 An 8 B reasoning VLM for physical AI

Cosmos Reason 2 is NVIDIA's open vision-language model purpose-built for "physical AI" — robots, autonomous vehicles, and any agent that has to *act* in the real world. Where most VLMs are trained on web-scale image-text pairs, Cosmos Reason 2 is post-trained on physical-AI datasets: ego-centric video, sensor logs, robotics trajectories, AV scenarios. The result is a model that has learned not just *what's in* a scene but how the scene's elements behave in time, and what the consequences of an action would be.

Cosmos Reason 2 ships in three sizes (2 B, 8 B, 32 B). Alpamayo 1.5 uses the **8 B variant** as its backbone — large enough to handle long video context and complex reasoning, small enough to fit alongside the action decoder on a single H100.

3.2 Architecture: ViT + dense transformer



[think] / [answer] denote the literal <think> and <answer> XML tags Cosmos emits

Figure 3.1 — Cosmos Reason 2 (8 B) architecture. ViT encodes images into visual tokens; a dense LLM transformer (post-trained from Qwen3-VL-8B-Instruct) processes the joint visual+text token stream and emits structured reasoning followed by a final answer. Inside Alpamayo, the "answer" stream feeds the diffusion action decoder.

WHAT "POST-TRAINED FROM QWEN3-VL-8B-INSTRUCT" MEANS

Cosmos Reason 2 did not start from scratch. NVIDIA took the open-source *Qwen3-VL-8B-Instruct* model — already a strong general-purpose vision-language model from Alibaba — and continued training it on physical-AI-specific data (autonomous driving clips, robotics trajectories, ego-centric video). The post-training is what specializes it: the same architecture, but the weights have been pulled toward physical reasoning.

3.3 Why Alpamayo uses Cosmos as its backbone

You could in principle build a driving VLA on any large vision-language model. Cosmos Reason 2 has three properties that made it the right choice for Alpamayo:

- **Physical-AI grounding:** it has already been exposed to driving and embodied scenarios during post-training, so far less of Alpamayo's downstream training has to teach it physics from first principles.
- **Long context (256 K tokens):** a multi-camera 0.4-second history of 4 frames per camera plus the chain-of-thought prompt is a lot of tokens. Cosmos Reason 2 can hold all of it.
- **Native chain-of-thought formatting:** it was trained to emit `<think>` tags. That makes Alpamayo's reasoning-first-then-action structure clean to wire up — no prompt engineering gymnastics.

Source: *Cosmos-Reason2-8B model card*; *cosmos-reason2 GitHub*.

4. AlpaSim — The Closed-Loop Testbed

4.1 Why simulate, and what kind of simulation?

You cannot evaluate an autonomous-driving model the way you evaluate, say, an image classifier. Driving is *closed-loop*: the model's action at time t changes what it sees at time $t+1$. A trajectory dataset can tell you how close the model's predictions match a recorded human driver, but it cannot tell you whether the model would actually *drive* the same scene end to end without crashing.

AlpaSim solves this by replaying real driving scenes in closed loop. Each scene is a 20-second clip from the NuRec dataset — a high-fidelity 3D reconstruction of an actual recorded drive. AlpaSim hands the recorded sensor stream to the driver model, lets the driver issue trajectories, integrates physics, and re-renders the scene from the driver's resulting pose. The other agents in the scene (cars, pedestrians) replay their recorded trajectories regardless of what the ego does. So if AlpaMayo nails the trajectory, the scene unfolds normally; if AlpaMayo deviates, the world stays where it was, and AlpaSim's eval pipeline reports the divergence.

4.2 Microservice architecture

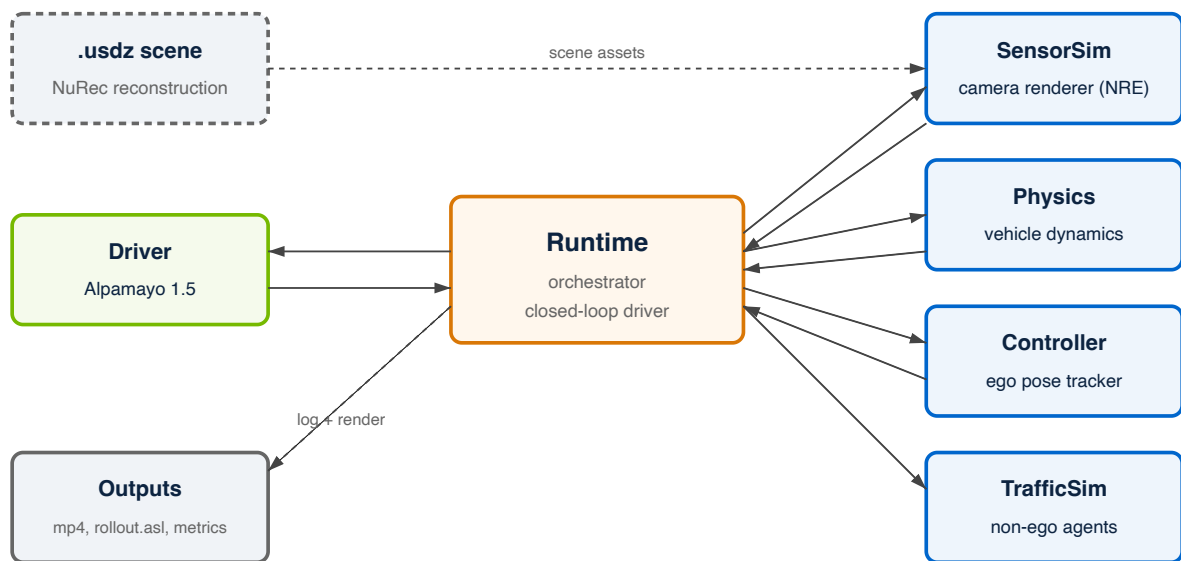


Figure 4.1 — AlpaSim's microservice topology. Each box is a separate Docker container. The Runtime orchestrates a closed-loop step: it asks SensorSim for the current camera frames, asks the Controller for the ego's egomotion history, sends both to the Driver, gets back a trajectory, runs Physics + TrafficSim to advance the world, and logs the step.

THE CLOSED-LOOP STEP, IN ORDER

1. Runtime fires a *StepEvent* at the current sim time.
2. SensorSim renders the current camera views from the ego pose using the USDZ scene assets and returns RGB frames.
3. Controller returns the ego's recent egomotion history.
4. Runtime packs (frames, egomotion, command) and sends to the Driver via gRPC.
5. Driver (Alpamayo 1.5) returns a 6.4-second trajectory plus reasoning text.

6. Physics integrates the first 100 ms of that trajectory, applying road-surface constraints.
7. TrafficSim advances the recorded non-ego agents by 100 ms.
8. Runtime logs the step (poses, decision, reasoning) into `rollout.asl`.
9. Sim time advances 100 ms; the cycle repeats. A 20-second scene = 200 cycles.

4.3 NuRec scenes: real driving made replayable

NuRec ("NeUral REConstruction") is NVIDIA's open dataset of high-fidelity 3D reconstructions of real driving clips. Each clip is a 20-second drive captured by an NVIDIA test vehicle in one of 25+ countries, then reconstructed into a USD-format 3D scene that AlpaSim can re-render from arbitrary ego poses. The dataset ships with per-scene metadata labels (`behavior`, `layout`, `weather`, `vrus`, `traffic_density`) that let you filter for the kinds of scenes you want — construction zones, intersections with pedestrians, highway merges, etc.

The current public release contains hundreds of scenes across the 25.07 and 26.02 batches, all gated on HuggingFace and downloaded on demand by the AlpaSim wizard.

4.4 Outputs: video, ASL, metrics

After a closed-loop run, AlpaSim emits:

- `rollout.asl` — a length-delimited protobuf log of every gRPC message exchanged. Includes per-step driver requests, driver returns (with reasoning text in the unstructured debug field), traffic, physics, controller poses. This is the canonical record of what happened.
- Per-rollout `metrics.parquet` and aggregate `metrics_results.txt` — collision rates, off-road, distance to ground-truth trajectory, plan deviation, progress.
- Two video layouts:
 - **DEFAULT**: bird's-eye-view map + camera with green/yellow planned vs ground-truth trajectory + Command label + scrolling metrics table.
 - **REASONING_OVERLAY**: front-camera with a cyan "Reasoning:" banner refreshing every 1 s, and a History/Prediction trajectory chart on the right.
- Per-service log files for every microservice (driver, controller, physics, runtime, sensorsim).

Source: [AlpaSim Tutorial](#).

5. How They Fit Together — A Single Driving Step

The clearest way to internalize all three pieces is to walk through one closed-loop step in detail. Here is what happens between sim-time t and $t + 100\text{ ms}$:



Figure 5.1 — One closed-loop step, end to end. Total wall-clock time per step on an H100: 100-200 ms for the model, plus a few ms for sim integration. Sensorsim warmup happens once per scene ($\approx 100\text{ s}$).

Two things to note about this loop:

- **The reasoning is not post-hoc.** Steps 3 and 4 happen in the same forward pass — the action decoder is conditioned on the same latents that produced the reasoning tokens. The model cannot decide an action and then make up an explanation; the explanation *is* the planning state.
- **The world is not reactive.** The other cars in the scene replay their recorded trajectories regardless of what the ego does. This is what makes NuRec scenes useful as benchmarks — when AlpaMayo deviates from the recorded driver's path, you can directly observe the consequences.

6. A Real Run, End to End

The screenshots below are from two scenarios run on Alpamayo 1.5 in AlpaSim, captured from this project's reproduction artifacts.

6.1 Scene one — construction zone (NVIDIA's published showcase)

Scene ID `clipgt-02eadd92-...` from NuRec — an urban drive with a slow lead truck, a red traffic light, parked vehicles encroaching from the curb, and a pedestrian crossing. This is the same clip NVIDIA features in Figures 1, 2, and 11 of their developer blog.

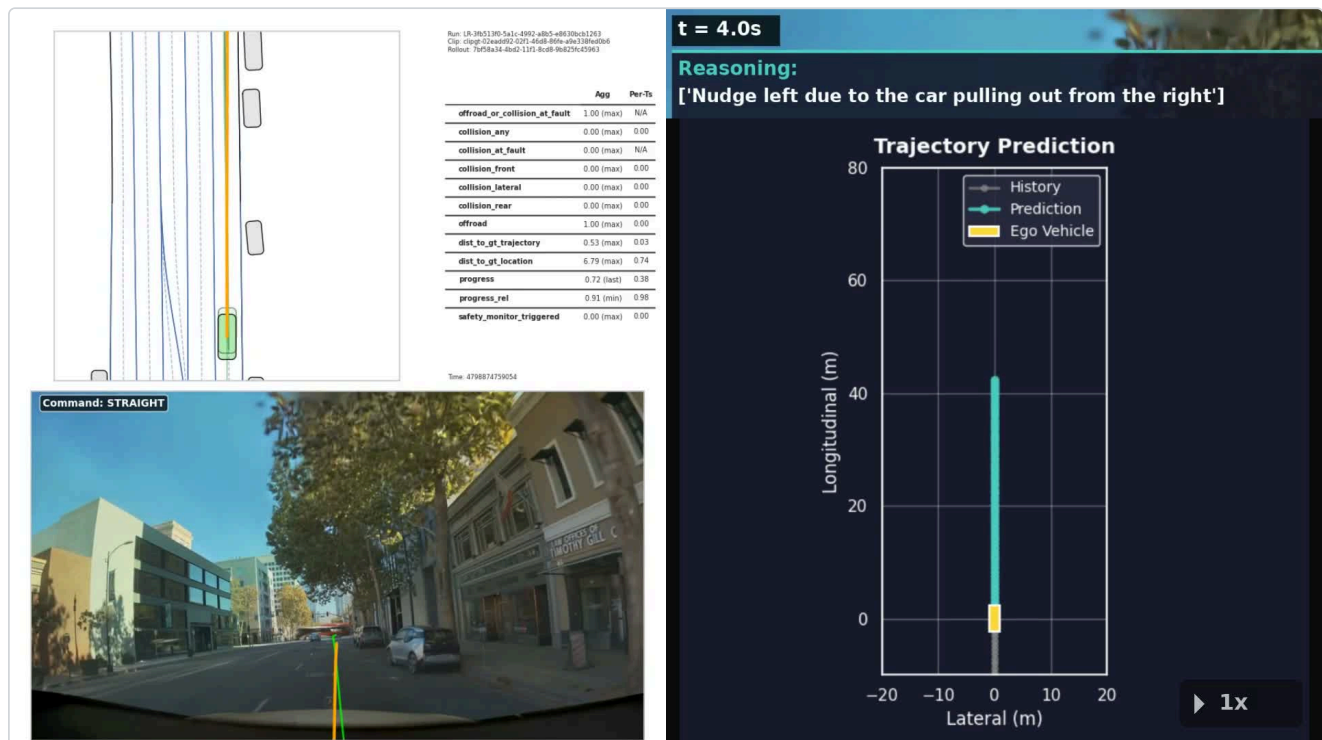


Figure 6.1 — Composite frame. Left: bird's-eye-view map + scrolling metrics table + camera with green (predicted) and yellow (ground-truth) trajectory and "Command: STRAIGHT" banner. Right: cyan reasoning banner and the History/Prediction trajectory chart. The reasoning at this moment reads: "Stop to yield to the pedestrian in the crosswalk ahead since they are walking across our path."

WHAT THE MODEL CONSIDERED (SAMPLE OF UNIQUE REASONING CHAINS ACROSS THE RUN)

- "Stop to yield to the pedestrian in the crosswalk since they are crossing ahead"
- "Nudge left due to the stopped truck blocking the right side of our lane"
- "Slow down to create a gap for a merge to the left because our current right lane narrows / ends into a parking / turn area ahead while a bus is occupying the left lane"
- "Stop for the red traffic light since the signal ahead is red"
- "Stop since bollards block the lane ahead"

The same model emitted ~30 distinct reasoning chains across a 20-second window, each tied to a 100 ms decision step.

6.2 Scene two — urban intersection with pedestrian crossing

Scene ID `clipgt-7b186d92-...` — selected by filtering NuRec's labels for `layout=intersection,pedestrian_crossing`, `vrus=true`, `weather=clear/cloudy`, `surface=dry`, `lighting=daytime`. Daytime urban intersection with stop signs, bollards, and a marked crosswalk with active pedestrians.

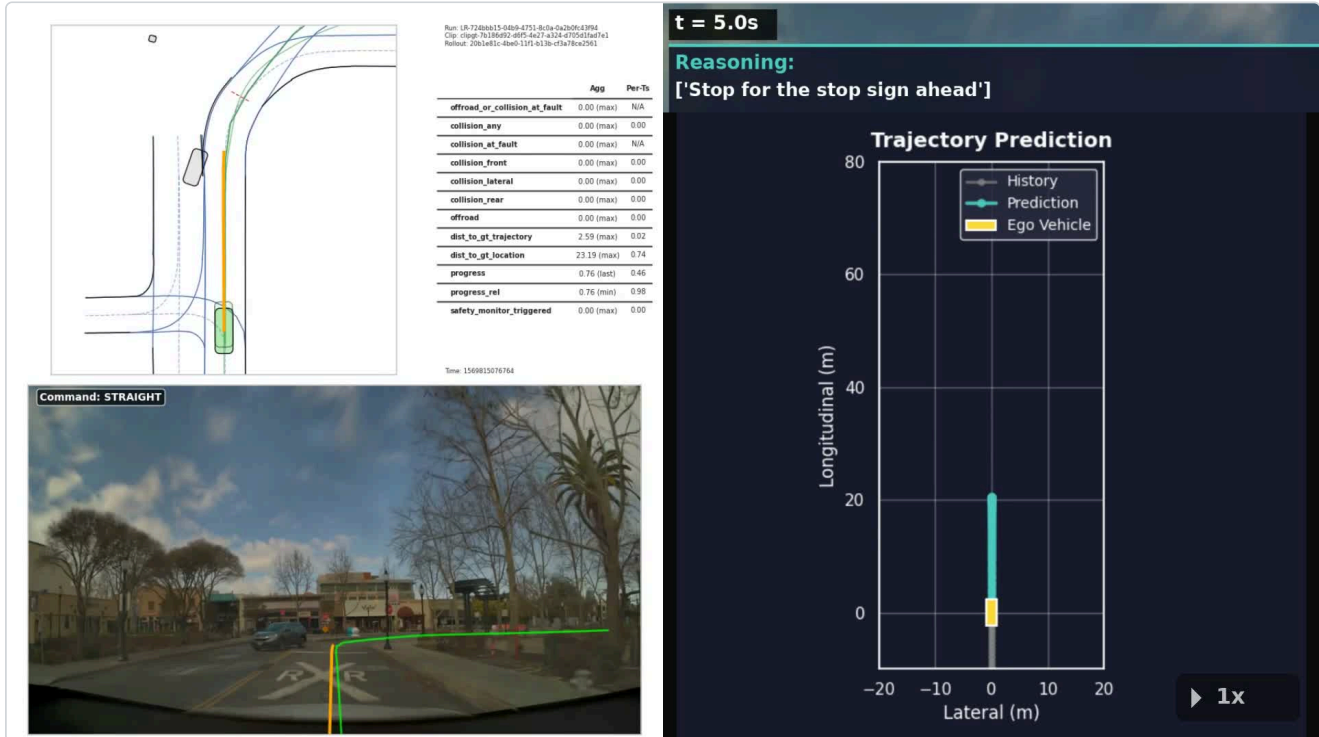


Figure 6.2 — Composite frame from the pedestrian-crossing scenario. The model navigated 70 m in ~18 s with no collisions; eval reported max 2.59 m deviation from the recorded ground-truth trajectory.

SAMPLE REASONING CHAINS

- "Stop for the stop sign controlling the intersection"
- "Yield due to a pedestrian walking across the crosswalk"
- "Turn right because the pedestrian has cleared our path"
- "Turn right because the one-way sign indicates right only"
- "Stop because a curbside bollard blocks the lane ahead"
- "Turn left due to the median blocking straight ahead"

Notice the diversity: the model distinguishes *why* it stops (pedestrian vs stop sign vs bollard vs red light) and *why* it turns (one-way sign vs median blocking the straight path). That fine-grained causal vocabulary is the contribution of the chain-of-causation training data — 3 million traces in Alpamayo 1.5's training set.

6.3 What the run produces, on disk

After AlpaSim finishes, the output directory for one scenario contains:

```
scenario_construction/
  rollouts/clipgt-02eadd92-.../<run-uuid>/
    rollout.asl                      368 MB binary protobuf log
    *_camera_front_wide_120fov_default.mp4
```

```
*_camera_front_wide_120fov_reasoning_overlay.mp4
metrics.parquet
aggregate/
  metrics_results.txt          eval summary
  metrics_results.parquet
  videos/all/00_all_clips_fast.mp4
controller/ driver/ telemetry/ txt-logs/
driver-config.yaml eval-config.yaml docker-compose.yaml ...
```

The `rollout.asl` is the canonical record. Each entry is a length-delimited protobuf `LogEntry` message; the `driver_return` variants carry the reasoning text in their unstructured debug field. Parsing it requires the `AlpaSim` utilities (or the ad-hoc Python recipe in this project's repository).

7. References

All technical claims in this guide are sourced from publicly available NVIDIA documentation and arXiv papers. Where specific numbers are quoted, the source is on the same page.

NVIDIA documentation

- Alpamayo 1.5 model card: huggingface.co/nvidia/Alpamayo-1.5-10B
- Alpamayo R1 (1) model card: huggingface.co/nvidia/Alpamayo-R1-10B
- Cosmos Reason 2 (8 B) model card: huggingface.co/nvidia/Cosmos-Reason2-8B
- NVIDIA developer blog — "Building AVs that Reason": developer.nvidia.com/blog/building-autonomous-vehicles-that-reason-with-nvidia-alpamayo
- AlpaSim repository: github.com/NVlabs/alpasim
- AlpaSim tutorial: github.com/NVlabs/alpasim/blob/main/docs/TUTORIAL.md
- Alpamayo source: github.com/NVlabs/alpamayo
- Cosmos Reason 2 source: github.com/nvidia-cosmos/cosmos-reason2
- NuRec dataset: huggingface.co/datasets/nvidia/PhysicalAI-Autonomous-Vehicles-NuRec

Papers

- Alpamayo R1: "Bridging Reasoning and Action Prediction for Generalizable Autonomous Driving in the Long Tail." arXiv 2511.00088. arxiv.org/abs/2511.00088
- Sim2Val (closed-loop validation): arXiv 2506.20553
- RoaD: arXiv 2512.01993
- CAT-K traffic model: arXiv 2412.05334

Background reading

- Qwen3-VL-8B-Instruct (Cosmos Reason 2's base model): huggingface.co/Qwen/Qwen3-VL-8B-Instruct
- Boris Ivanovic's keynote — Berkeley Robotics & AI Conference, April 29, 2026 (the talk that motivated this study).

About the diagrams in this guide

All architectural diagrams (Figures 2.1, 3.1, 4.1, 5.1) were drawn for this study guide as inline SVG. They synthesize information from the cited model cards and the AlpaSim source code; they are not reproductions of NVIDIA's official figures. The composite stills in Figures 6.1 and 6.2 are screenshots from the author's own AlpaSim runs.